documents are difficult to maintain. A tool such as *doxygen* is useful for generating source code documentation.

Use a coding checklist

A self-review coding checklist must be maintained by the developer. The checklist should be updated as and when defects are found. And it should be updated with actionable checklist points, which could ensure that any defect is not introduced. Using the checklist would ascertain that the same defect is not repeated. All members in the team should use that checklist and update it periodically.

Analysing the '5 Whys'

Despite the quality processes you adopt, you must admit the fact that defects do somehow creep in. After all, software developers are only human. So when you find a defect, what should you do with it? Most software developers would be tempted to 'fix and close it'.

But isn't it important to find out what caused the defect? Yes, it is. Analysing the cause of a defect can let you catch some other defects that may exist, and prevent similar defects being introduced in the future.

I have found the '5 Whys' method [as described in the book ***Kaizen: The Key to Japan's Competitive Success*** by Masaaki Imai (*www.amazon.com/Kaizen-Key-Japans-Competitive-Success/dp/007554332X*)], to be extremely useful to reach the root cause of the defect.

The '5 Whys' method involves the asking of questions to identify the root cause of a problem. For example, if a person dies due to an accident, the objective is to identify why he died. So, we put up a chain of questions, until we reach the root cause of the problem. This is done as follows:

John died. [The Problem]

1. Why? – He met with an accident.
2. Why? – He was driving his car quite fast.
3. Why? – The car brakes were not working.
4. Why? – The brake oil was not checked for a long time, and hence brakes failed. [The root cause has been found in the '4th Why', itself]

There need not always be five questions to be asked. Sometimes, you may need to keep on questioning until you reach the root cause. And in some cases, you might end up reaching the root cause in just three questions! The idea behind this method is to drill-down to reach the root cause of the problem.

Knowing the root cause of the defect can help you to prevent similar defects in future.

Frequent releases

You need to plan for frequent releases, because releases at regular intervals maintain the tempo of the team. At the same time, you get a chance to refine your processes based on the feedback and the lessons learnt. It would also ensure that there is no slippage in the schedule.

Tests and testability

Write code that is testable. If your code is not testable, it implies complex code, and hence poor maintainability and higher chances of defects.

Do the testing, starting with the assumption that there are defects. Take different input combinations. Take not only the positive patterns, but the negative patterns as well. List down the following when you create a test case:

- Purpose of the test case [what it intends to test]
- Expected output
- Hardware/software requirements

For developers, it is highly important to test whatever they code. Performing unit testing is an effective method to find defects in your code. See to it that your code can be isolated from the rest of the code for unit testing.

In a team, we have several developers working on different modules. The challenge is to seamlessly integrate all the changes to form a single product. Often, we fail to analyse the impact of changes in one module with respect to other modules. What I have experienced is that many defects creep in due to interface problems. Defects come in when an interface is either not tested at all, or is not tested fully. So, the interface must be tested while integrating changes in a module with the rest of the modules.

While you test, check whether your code is doing what it is supposed to do. More importantly, check whether it is doing what it is not supposed to. If it is doing something it should not be doing, that's a defect.

Some handy tips

By catching defects early, the amount of time spent in fixing things later is reduced. Keep the following steps in mind to prevent defects, and catch them as early as possible:

- Avoid introducing new defects in the code.
- Use a checklist.
- Code should be simple, as simple code is easy to maintain.
- Avoid global variables and magic numbers.
- Complex design leads to poor testability.
- The user should be central to the design.
- Test as you code.
- Change of code should support ease of testing, debugging and modification.

As in everything else, prevention is better than cure. So prevent the defects in your program, and you won't need to waste resources on finding them. END

By: Deepti Sharma

The author is a senior software engineer (team lead) at Acme Technologies. She has considerable amount of experience in the development of compilers and tools for embedded systems. She can be reached at deepti DOT acmet AT gmail DOT com.